

# 修了試験B-4

合計点 22/28 点 ?

合格点 85%以上 (24以上/28)

0 / 0 ポイント

## 受験者情報の登録

登録メールアドレス \*

htanaka@777.so-net.jp

御名前 \*

田中 秀伯

## 修了試験問題

22 / 28 ポイント

- ✓ 近年、エッジコンピューティングの発展に伴い、IoTデバイスなどが広く \*1/1  
利用されている。エッジコンピューティングにおいて、IoTデバイスで使  
用する深層学習モデルの軽量化が重要な理由について、以下のうちから  
適切なものを1つ選べ。
- ☐ IoTデバイスは高性能なプロセッサを持っているため、より大きなモデルを動かす  
ことができる。
- ☐ IoTデバイスは常にサーバーに接続されているため、モデルのサイズはデバイスの  
性能に依存しない。
- ☐ IoTデバイスはバッテリー駆動のため、大きなモデルはバッテリー寿命を短くする  
が、処理速度には影響しない。
- ☒ IoTデバイスのモデルサイズや消費電力には限界があるため、精度を落とさず ✓  
に必要なメモリ使用量を小さくする必要がある。



- ✓ IoTデバイスで動かすことのできる深層学習モデルには、モデルサイズ、\*1/1 計算量、メモリ使用量、消費電力などの制約がある。そのため、精度低下を抑えながらモデルを軽量化する必要がある。モデルの軽量化に関する記述として、最も適切なものを1つ選べ。
- ☐ 効率的なマイクロアーキテクチャの設計とは、訓練中にモデルの層数を動的に増減させる手法のみを指す。
  - ☐ 量子化は、モデルを構成するすべての層を削除することで、モデルサイズや計算量を削減する手法である。
  - ☐ 知識蒸留は、教師モデルのパラメータをそのまま生徒モデルへコピーする手法である。
  - ☒ プルーニングは、重要度の低い重み、フィルタ、チャネルなどを削減し、モデルサイズや計算量を小さくする手法である。 ✓

現実の環境下では、計算資源の制約が存在し、モデルの軽量化が必要となる場面が存在する。その際、出来るだけモデルの性能を低下させずにモデル容量を小さくする軽量化手法が求められる。軽量化には様々な観点から手法が提案されている。

例えば、データを少ないビット数で表現することで計算の高速化・省メモリ化を図る手法を（ あ ）と呼ぶ。活性化を二値化するとほとんどの勾配が0になる。これを防ぐために、逆伝播時に活性化関数を恒等写像関数として扱う STE（Straight-Through Estimator）という処理を行うことがある。

（ い ）は、大きなモデルで得た知識を小さなモデルに移管する手法である。エッジデバイスなど、限られた計算資源のなかで動作することを目的として、（ う ）のための処理といえる。学習時と推論時で（ え ）モデルを使うことになる。

✓ ( あ ) について、以下のうちから適切なものを1つ選べ。 \*

1/1

- ☒ 量子化
- ☐ 剪定
- ☐ 分散
- ☐ 蒸留



✓ ( い ) について、以下のうちから適切なものを1つ選べ。 \*

1/1

- ☐ 陳腐化
- ☒ 蒸留
- ☐ プルーニング
- ☐ 量子化



✓ ( う ) と ( え ) について、以下のうちから適切なものを1つ選べ。 \*1/1

- ☐ (う) 精度向上 (え) 異なる
- ☒ (う) 処理速度向上 (え) 異なる
- ☐ (う) 処理速度向上 (え) 同じ
- ☐ (う) 精度向上 (え) 同じ



モデルの軽量化では、精度と計算量のバランスを考慮しながら、モデル構造や演算方法を工夫することが重要である。適切な軽量化手法を利用することで、精度低下を抑えながら、モデルサイズや計算量を削減できる場合がある。

代表的な軽量化手法の一つに、重要度の低い重み、フィルタ、チャネルなどを削減する（ あ ）がある。Heらが2017年に提案したチャネルプルーニング手法では、チャネル選択に（ い ）が用いられた。削減後に（ う ）ことで、精度を回復できる場合がある。一方、削減すべき要素を学習前に正確に特定することは（ え ）。

✓ （ あ ）の選択肢について、以下のうちから適切なものを1つ選べ。 \* 1/1

- ☐ 蒸留
- ☒ プルーニング
- ☐ 陳腐化
- ☐ 量子化



✓ （ い ）の選択肢について、以下のうちから適切なものを1つ選べ。 \* 1/1

- ☐ Ridge回帰
- ☒ LASSO回帰
- ☐ テイラー展開
- ☐ Elastic Net



✓ ( う ) と ( え ) の選択肢について、以下のうちから適切なものを1つ選べ。 \*1/1

- ☐ (う) ファインチューニングしない (え) 一般に容易である
- ☒ (う) ファインチューニングする (え) 一般に容易ではない ✓
- ☐ (う) ファインチューニングしない (え) 一般に容易ではない
- ☐ (う) ファインチューニングする (え) 一般に容易である

✓ 大規模なモデルを複数のデバイスに分割し、デバイス間で中間出力や勾配を受け渡ししながら学習する手法をモデル並列という。モデル並列に関する記述として、誤っているものを1つ選べ。 \*1/1

- ☐ モデル並列化の実装には、モデルのどの部分をどのマシンで処理するかを慎重に計画する必要があり、これにはタスクの性質やマシンの能力を考慮することが重要である。
- ☒ モデル並列化は主に、データセットのサイズが大きい場合にのみ使用され、モデルのサイズや複雑さには依存しない。 ✓
- ☐ モデル並列化では、大規模なモデルを複数のマシンに分散して処理することで、一つのマシンでは処理が困難なタスクを実行できる。
- ☐ モデル並列化では、モデルを上流と下流で分割する方法や、モデルの異なる部分を別々のマシンで処理するなどのアプローチがある。

✕ ディープラーニングの規模が大きくなると、一台のパソコンでは処理することが困難という事態が発生する。その対策手法として並列分散学習がある。並列分散学習の1つであるデータ並列について、以下のうちから適切なものを1つ選べ。 \*0/1

- ☐ データ並列処理では、異なるデバイス上で異なるモデルが異なるデータを同時に処理する。各デバイスは独立して学習し、学習結果は終了後に結合される。
- ☐ データ並列処理では、複数のデバイス上で同一のモデルが異なるデータを処理し、全てのデバイスが同一のパラメータを共有する。
- ☐ データ並列処理では、一つのデバイス上で複数のモデルが同時に異なるデータセットを処理し、各モデルは独自の更新スケジュールに従う。
- ☒ データ並列処理では、複数のデバイス上で同一のモデルが異なるデータを処理するが、各デバイスは異なるパラメータを使用し、これによりモデルの多様性が保証される。 ✕

AIの実用化にはモデル知識だけでなく、深層学習ライブラリの計算グラフ構築には、事前に計算グラフを定義してから実行する「Define and Run」と、実行時に処理内容に応じて計算グラフを動的に構築する（ あ ）がある。代表的に（ あ ）を採用する深層学習ライブラリとして（ い ）が挙げられる。なお、TensorFlow 2ではEager Executionがデフォルトで有効になっており、必要に応じてtf.functionを用いたグラフ実行も利用できる。

GPUは科学計算やAI学習における高速化に貢献しているプロセッサの一種である。モデル並列化は特に（ う ）の効率的な計算で使われる学習加速手法である。データ並列化は同期型と非同期型があり、同期型は非同期型に比べて（ え ）が、計算コストがかかる傾向がある。

✓ (あ)と(い)の選択肢について、以下のうちから適切なものを1つ選べ。 \*1/1

- ☐ (あ)「Define by Run」 (い) PyTorch
- ☒ (あ)「Define by Run」 (い) TensorFlow ✓
- ☐ (あ)「Define or Run」 (い) PyTorch
- ☐ (あ)「Define or Run」 (い) TensorFlow

✓ (う)と(え)の選択肢について、以下のうちから適切なものを1つ選べ。 \*1/1

- ☐ (う)大量のデータセット (え)計算速度が速い
- ☒ (う)大規模モデル (え)精度が高い ✓
- ☐ (う)大量のデータセット (え)精度が高い
- ☐ (う)大規模モデル (え)計算速度が速い



( あ ) は、各プロセスに同一のモデルを配置し、それぞれが異なるミニバッチを処理した後、勾配を集約してパラメータを更新する手法である。( い ) は、1つのモデルを複数の部分に分割して複数のプロセスへ配置し、各プロセスが協調して1つのモデルを学習する手法である。ここで、図1は ( う )、図2は ( え ) の概念図を表す。

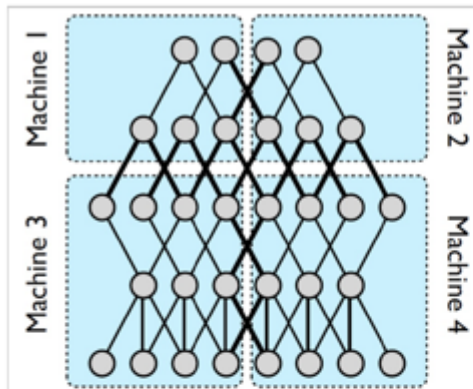


図1: ( う ) の概念図 (Dean Jeffrey, et al., "Large scale distributed deep networks.", 2012. Figure 1 より抜粋)

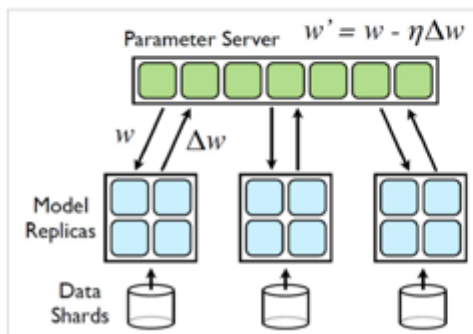


図2: ( え ) の概念図 (図1 同論文 Figure 2 より抜粋)

✕ ( あ ) ( い ) ( う ) ( え ) に当てはまる語句の組み合わせ \*0/1  
として、適切なものを1つ選べ。

- ☐ (あ) モデル並列化 (い) データ並列化 (う) データ並列化 (え) モデル並列化
- ☐ (あ) モデル並列化 (い) データ並列化 (う) モデル並列化 (え) データ並列化
- ☒ (あ) データ並列化 (い) モデル並列化 (う) データ並列化 (え) モデル並列化 ✕
- ☐ (あ) データ並列化 (い) モデル並列化 (う) モデル並列化 (え) データ並列化



データ並列化のパラメータ更新方式には、同期型と非同期型がある。非同期型では、各ワーカーが他のワーカーの更新完了を待たずにパラメータを更新するため、同期型よりもスループットが（ お ） 場合がある。

一方、図3の実験では、（ か ） の方がより低い負の対数尤度に到達している。

図3は、PixelCNNモデルの学習過程を負の対数尤度（Negative Log-Likelihood）で評価したものである。

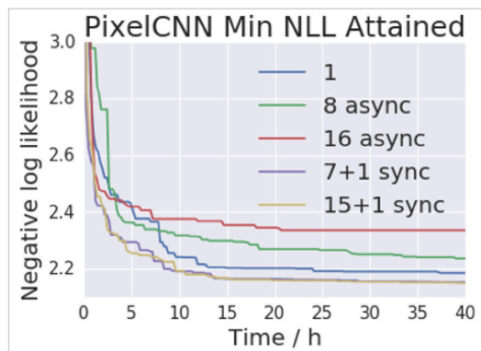


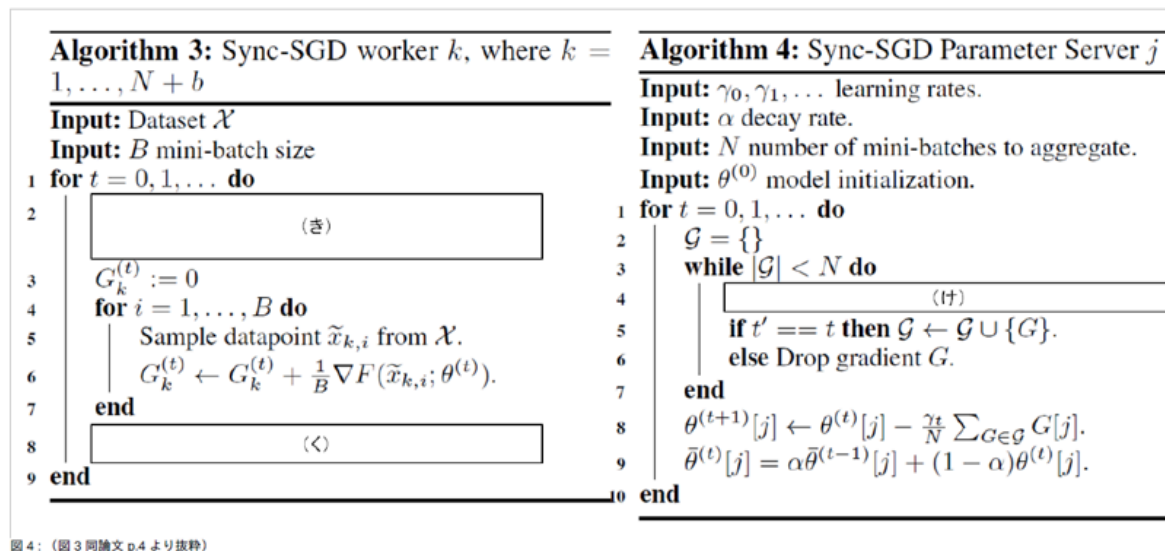
図 3：（Chen Jianmin, et al., "Revisiting distributed synchronous SGD.", 2016. Figure 9 より抜粋）

✓ （ お ） （ か ） に当てはまる語句の組み合わせとして適切な選択肢 <sup>\*</sup>1/1 を 1 つ 選べ。

- ☒ （お）高い （か）同期型
- ☐ （お）低い （か）非同期型
- ☐ （お）高い （か）非同期型
- ☐ （お）低い （か）同期型



図4は同期型のデータ並列型のアルゴリズムを表している。左図がワーカー、右図がパラメータサーバ（パラメータを管理するサーバ）のアルゴリズムである。



( き ) ( く ) ( け ) には、以下の処理のいずれかが入る。

処理 A: Wait to read  $\theta^{(t)} = (\theta^{(t)}[0], \dots, \theta^{(t)}[M])$  from parameter servers.

処理 B: Send  $(G_k^{(t)}, t)$  to parameter servers.

処理 C: Wait for  $(G, t')$  from any worker.

✓ ( き ) ( く ) ( け ) に当てはまる処理の組み合わせとして、\*1/1 適切なものを1つ選べ。

- ☐ (き) 処理C (く) 処理B (け) 処理A
- ☐ (き) 処理A (く) 処理C (け) 処理B
- ☐ (き) 処理B (く) 処理C (け) 処理C
- ☒ (き) 処理A (く) 処理B (け) 処理C



## ✓ CPUとGPUに関して適切でないものを1つ選べ。 \*

1/1

- ☐ DNNの計算をGPUで行うときには、単精度浮動小数点数で計算されることが多い。
- ☒ CPUとGPUが計算に利用するメモリに関して、CPUの方が単位時間当たりのデータ読み出し量が重要視される。 ✓
- ☐ CPUに比べて、GPUはコア当たりの計算性能が低い。
- ☐ GPUは行列演算のように単純な計算を並列に複数回行うような処理を得意とする。

## ✓ CPU、GPU、TPUおよびGPGPUに関する記述として、誤っているものを1つ選べ。 \*1/1

- ☐ GPGPUは、GPUの演算能力をグラフィックス処理以外の科学計算やデータ解析などに利用する技術である。
- ☒ TPUは汎用的な逐次処理を主な目的として設計されており、機械学習で多用される行列演算の高速化には特化していない。 ✓
- ☐ GPUは多数の演算器を備え、同じ種類の計算を大量のデータに対して並列に実行する処理を得意とする。
- ☐ CPUは汎用性が高く、複雑な制御や条件分岐を含むさまざまな処理に利用される。

✓ グラフィック処理に特化して開発されたGPUが、科学技術計算や深層学習などの分野で広く用いられるようになった経緯と、その利点や欠点について考えることは、現代のコンピューターサイエンスにおいて重要な視点である。GPUの使用における説明文について、以下のうちから適切なものを1つ選べ。 \*1/1

- ☐ GPUは初めから多目的に使用可能なように設計されており、エラー検出と訂正機能も備えている。これにより科学計算や深層学習での利用が可能となったが、その性能は専用の科学計算チップに比べると劣る。
- ☐ GPUはもともと映像描画のために開発されたが、エラー検出と訂正の機能が付いているため、科学計算や深層学習で広く使われている。しかし、その高い演算性能はコストが高いため、開発には適さない。
- ☐ GPUは当初から科学計算用に設計されたが、後にグラフィック処理にも利用されるようになった。その高い演算性能は低コストでの開発に貢献しており、メモリエラー検出機能が標準装備されているため、科学計算にも安全に使用できる。
- ☒ GPUは元々、ゲームや3Dグラフィックスの描画処理向けに発展したが、その高い並列演算性能をグラフィックス以外にも活用することで、科学技術計算や深層学習にも広く用いられるようになった。 ✓

✓ コンピュータの並列処理アーキテクチャには大きく分けて4つの種類があり、これらはフリンの分類と呼ばれている。それぞれの並列処理アーキテクチャの説明文について、以下のうち誤っているものを1つ選べ。 \*1/1

- ☒ MIMDは複数のプロセッサが異なる命令を異なるデータに対して同一の処理を同時に行う。分散システムなどに使用される。 ✓
- ☐ MISDは複数の命令が同時に一つのデータに対して実行される。このタイプは非常に珍しく、特定の専用システムでのみ見られる。
- ☐ SISDは伝統的なシングルコアプロセッサで見られるアーキテクチャで、一つの命令が一つのデータポイントに対して実行される。
- ☐ SIMDは一つの命令で多数のデータに対して同時に同じ操作を行う。画像処理などデータが一様な処理に適している。

✕ 仮想化技術について、以下のうち正しい選択肢を1つ選べ。 \*

0/1

- ☒ コンテナ型仮想化は、ホスト型仮想化やハイパーバイザー型仮想化よりも、一般に大きなオーバーヘッドを持つ。 ✕
- ☐ コンテナ型の代表ソフトウェアとして、Hyper-V がある。
- ☐ ホスト型仮想化では、複数の仮想マシンがホストOSのカーネルを共有して動作する。
- ☐ ハイパーバイザー型仮想化では、ハイパーバイザーがハードウェア上で直接動作し、仮想マシンを管理する。

✓ Docker について、以下のうち誤っている選択肢を1つ選べ。 \*

1/1

- ☐ Docker Hub では、自身で作成した Docker イメージを保管するだけでなく、他のユーザーが作成した Docker イメージを利用することができる。
- ☒ ユーザーがdocker pullを実行すると、Docker client自身がDocker daemonを介さずにレジストリからイメージを直接取得し、ローカルへ保存する。 ✓
- ☐ Docker は、コンテナ型のアプリケーション実行環境を管理するためのオープンソースソフトウェアであり、Linux のコンテナ技術に基づいて設計されている。
- ☐ Docker daemon は、Docker client から Docker API を介してリクエストを受け取り、Dockerオブジェクトを管理する。

✓ 仮想化に関して誤っているものを1つ選べ。 \*

1/1

- ☒ OSレベル仮想化の方が、ハードウェアレベル仮想化と比べると、ゲストOSの管理が必要なため、管理コストが高い。 ✓
- ☐ OSレベル仮想化では、アプリケーション実行環境を分割した「隔離環境」を利用する。
- ☐ ハードウェアレベル仮想化の方が、OSレベル仮想化と比べると、利用資源が多いため、性能が出にくいとされている。
- ☐ ハードウェアレベル仮想化は、仮想化ソフトウェアを利用してサーバを疑似的に再現した仮想マシンを提供する。



✓ 各仮想化方式に関する記述として、誤っているものを1つ選べ。 \*

1/1

- ☐ ホスト型仮想化では、ホストOS上で動作する仮想化ソフトウェアが仮想マシンを管理する。
- ☐ ハイパーバイザー型仮想化では、物理ハードウェア上で動作するハイパーバイザーが仮想マシンを管理する。
- ☐ ホスト型仮想化の例としてVirtualBox、ハイパーバイザー型仮想化の例としてVMware ESXi、コンテナ型の例としてDockerが挙げられる。
- ☒ コンテナ型仮想化では、コンテナごとに個別のゲストOSを起動し、その上でアプリケーションを実行する。 ✓

✕ ホスト型仮想化やハイパーバイザー型仮想化と比較した場合の、コンテナ型仮想化の特徴に関する記述として、誤っているものを1つ選べ。 \*0/1

- ☐ コンテナはホスト環境から完全に分離されるため、追加の設定を行わなくても、仮想マシンと同等以上の隔離性と安全性が常に保証される。
- ☒ 仮想マシンと比較して、メモリやストレージなどのコンピュータリソースの消費を抑えやすい。 ✕
- ☐ コンテナごとに個別のゲストOSを用意する必要がないため、一般に起動時間や管理負荷を抑えやすい。
- ☐ ホストOSのカーネルを共有することで、アプリケーションを比較的軽量に実行できる。

✕ Docker は、クライアントサーバシステムで構成されている。Docker を構成するものの機能について、説明したもののうち、誤っているものを1つ選べ。 \*0/1

- ☐ Docker daemonは、Docker APIのリクエストを受け取り、イメージ、コンテナ、ネットワーク、ボリュームなどのDockerオブジェクトを管理する。
- ☒ Docker registryは、実行中のコンテナやプロセスを保存し、その動作状態を管理する。 ✕
- ☐ Docker Hubは、Dockerイメージを保存・共有できるパブリックレジストリであり、Dockerが標準で参照するレジストリである。
- ☐ Docker clientは、ユーザーが入力したコマンドをDocker API経由でDocker daemonへ送信する。

✕ Dockerには、さまざまな操作を行うためのコマンドが用意されている。 \*0/1  
例えば、Dockerのバージョンを確認する場合は、docker versionを実行する。Docker Hub上の公開イメージを検索する場合は（ あ ）、レジストリからイメージを取得する場合は（ い ）、Dockerイメージを新たにビルドする場合は（ う ）を実行する。  
（ あ ）（ い ）（ う ）に当てはまるコマンドの組み合わせとして、適切なものを1つ選べ。

- ☐ （あ） docker search （い） docker get （う） docker create
- ☒ （あ） docker image （い） docker get （う） docker create ✕
- ☐ （あ） docker search （い） docker pull （う） docker build
- ☐ （あ） docker image （い） docker pull （う） docker build

- ✓ Dockerでは、コンテナイメージを構築するための命令を（ あ ）に記述する。一方、アプリケーションを構成するサービス、ネットワーク、ボリューム、起動設定などは（ い ）に記述できる。 \*1/1

（ あ ）（ い ）に当てはまる語句の組み合わせとして、適切なものを1つ選べ。

- ☒ （あ） Dockerfile （い） docker-compose.yml ✓
- ☐ （あ） docker.service （い） requirements.txt
- ☐ （あ） requirements.txt （い） docker.service
- ☐ （あ） docker-compose.yml （い） Dockerfile

- ✓ 以下は、PythonアプリケーションのDockerイメージを作成するためのDockerfileの例である。このDockerfile内で、明示的に指定されていないものを1つ選べ。 \*1/1

```
FROM python:3.12-slim
LABEL author="User"
COPY requirements.txt /requirements.txt
RUN pip install --no-cache-dir -r /requirements.txt
COPY server.py /server.py
ENV PORT=80
CMD ["python", "-u", "/server.py"]
```

- ☒ 作業ディレクトリの指定 ✓
- ☐ 環境変数の指定
- ☐ ベースイメージの指定
- ☐ Pythonパッケージのインストール

このフォームは デロイト トーマツ ディープスクエア株式会社 内部で作成されました。  
このフォームが不審だと思われる場合 [報告](#)

Google フォーム





